COL 764 Assignment 1: Sparse Retrieval

A from scratch inverted index with BM25+ and RM3

Aditya Anand — 2023CS50284

September 5, 2026

The submitted system is a C++ engine behind the three required Python entrypoints in `submission/retrieve.py`. All indexing, compression and scoring happen in C++; Python only dispatches, picks parameters from collection statistics, and gzips the finished index. A Python path (`indexer.py`, `boolean_vsm.py`, `bm25.py`, `custom_scorer.py`) is submitted alongside it, since the Boolean and vector-space components are graded on independent correctness and a small readable implementation is far easier to unit-test than the packed postings reader.

1 Indexing design

1.1 Analysis chain

Documents and queries are analysed by one function, `each_surface` in `analysis.cpp`, so a held-out query cannot possibly be tokenised differently from the collection it is scored against. Text is lowercased through a 256-entry byte table and split at every non-alphanumeric character. Single letters separated only by periods are then glued back together, turning `U.S.` into `us` and `F.B.I.` into `fbi`; without that rule an acronym decomposes into three length-1 tokens, and length-1 tokens are discarded. Stopping uses `INDEX_STOPWORDS`, a Lucene-style English list extended with question words, applied identically at index and query time. A second, much more aggressive Fox-style list (`STOPWORDS`) is never used for indexing; it only prunes the RM3 forward index on collections of at least 20,000 documents, where common verbs would otherwise dominate the expansion candidates. Stemming is Porter by default, using my own `PorterStemmer` rather than NLTK so it runs in the same thread as tokenisation, with results memoised in a thread-local cache; `IR_STEMMER=1` or `0` selects the S-stemmer or no stemming. Because query analysis must match whatever the index was built with, the stemmer and stopword flags go into `meta.bin` uncompressed and are read back in `load_index`.

The required Boolean/VSM/Python-BM25 path deliberately uses a simpler indexer (`indexer.py`: S-stemming, no stop list, no acronym folding), scored by the unit tests in `tests/test_retrievers.py` against hand-checked examples. Table 1 therefore compares two analysis chains on the same topics, which is what the assignment asks for: an independently verifiable Boolean/VSM retriever set against BM25, not one tokeniser held fixed across every row.

1.2 What is persisted

Five files are enough to reconstruct the scorer. Four of them are gzipped after the build if gzip actually helps (`postings.bin` at level 6, the rest at level 9); `meta.bin` is left plain because it is read before anything else is inflated.

- `dict.bin` holds the sorted vocabulary, front-coded as a vbyte shared-prefix length plus the suffix. A further vbyte packs df with the codec id and the `has_title` flag, and the postings offset is a delta. Term ids are array positions, so lookup is a binary search.
- `postings.bin` switches codec on list length. Below $df = 8$ a list is gap-plus-tf vbyte with the low bit flagging tf=1, which avoids a second byte on the overwhelming majority of rare terms. Longer lists use Simple-9 over $tf - 1$ alongside Elias–Fano over the doc ids, or `CODEC_S9EF_UNIT` when every body tf is 1 and the Simple-9 payload can be omitted entirely. Title frequencies ride along as a sparse vbyte sidecar. Long lists are never stored as vbyte even though vbyte is occasionally a few bytes smaller: decoding high-df query terms was the single largest term in query latency, and Elias–Fano decodes far faster.
- `docmap.bin` adapts to the external ids: if they share a length and an alphabet of at most 64 symbols they become base-A codes whenever that beats raw bytes, equal-length ids that fail the alphabet test are stored raw, and variable-length ids are front-coded.
- `doclens.bin` stores N, avgdl, the corpus token total, and one byte per document holding $|d|/\text{avgdl} = 64 \log_2(1 + r)$, code 255 saturating at 64. BM25 needs that ratio only to about a percent, so a float per document would waste three bytes each.

Several things are deliberately absent. RSJ IDF has a closed form and is computed at load, while collection frequency and the RM3 forward index are rebuilt in `load_index`, which costs nothing on the graded clock because the harness times build and not load. Positions are not stored at all: that is the one omission which visibly costs accuracy (query 31 in §4), and the first thing I would revisit if the index-size component were weighted lower.

1.3 Build path

`build_index_from_file` `mmap`s the JSONL file, splits it into line spans, and runs SPIMI over them. The JSON parser locates the `doc_id` and `text` keys directly and unescapes only when a backslash is present. At most four workers, matching the grading VM, each own a contiguous range of doc ids, and the count drops at 400k and 800k documents so the mapping plus the per-worker term maps stay inside 8 GB. Vocabulary ranges merge and encode

in parallel, but dictionary front-coding stays serial, so W=1 and W=4 emit byte-identical files; the caller thread acts as worker 0 rather than idling. Codec correctness is not left to inspection. `check_codecs` round-trips the fast `S9Reader` and `EFReader` against the reference unpackers over 2000 random lists plus every Simple-9 selector boundary, and checks that `s9_skip` leaves the cursor exactly where a full decode would.

2 Retrieval models

2.1 Boolean and vector space (required)

`boolean_search` intersects or unions posting sets and returns unranked, sorted doc ids. Its correctness is established by unit tests on a small corpus with hand-checked results rather than by nDCG, and the dev numbers show why. Over the 50 CORD-19 topics the conjunctive query returns a mean of 4 documents at recall 0.004, with 34 of the 50 conjunctions empty, while the disjunctive query returns a mean of 1.5×10^5 documents at recall 0.985. AND is far too strict for long natural-language topics and OR is very nearly the whole collection; ranking a lexicographic prefix of the OR set would produce a number but not a meaningful one, so Table 1 omits those rows.

`vsm_score` is `lnc.ltc` cosine with $w = (1 + \log tf) \cdot \ln(N/df)$ on both sides. The document norm $\|d\|$ is the L2 norm of the full document vector, computed at build time. Normalising over the query-term subspace instead is an easy mistake and a silent one: it scores every matching document exactly 1.0 on a single-term query, and over-rewards documents that miss a term on multi-term queries. Ties break by score descending, then doc id ascending, throughout the system.

2.2 BM25+ (required)

`bm25.py` and the C++ inner loop implement the same Okapi body with tunable k_1 and b, where δ=0 recovers textbook BM25. The C++ version adds query-tf normalisation and a title indicator:

$$w_q \left[\frac{tf_{\text{eff}}(k_1+1)}{tf_{\text{eff}} + k_1(1-b+b\,|d|/\text{avgdl})} + \delta + \beta \mathbb{1}[tf^{\text{title}} > 0] \right], \quad w_q = \frac{qtf_q}{\sum qtf} \ln\!\left(1 + \frac{N - df + 0.5}{df + 0.5}\right).$$

In the shipped configuration w_{title}=0, so tf_{eff} is the body frequency and the title enters only through the indicator β. The title window is the first 12 token *positions* after analysis, where a stopword consumes a position but is not stored, so the window tracks the original text instead of drifting as the stop list changes. Scoring is term-at-a-time into an accumulator tagged with a generation stamp, which avoids zeroing the array between queries; the top k come out of `nth_element` followed by a sort. The length factor $k_1(1-b+b\,|d|/\text{avgdl})$ is precomputed per parameter pair into a 256-entry table indexed by the quantised length byte, so the inner loop does one lookup instead of a divide.

2.3 Custom scorer and RM3 (competition)

`custom_scorer.py` min–max normalises Python BM25 (k_1=0.9, b=0.4) and cosine over a pool of 200 and fuses them at 0.85/0.15. Raw blending is not an option because BM25 is unbounded while cosine lies in $[0, 1]$. On CORD-19 the fusion is worse than Python BM25 alone (Table 1), so `retrieve()` does not call it; it is reported rather than quietly dropped.

The competition ranking is C++ BM25+ with optional RM3. From the top *fb_docs* of the first pass, $P(w|R) = \sum_i P(d_i|R)\,(tf_{w,d_i} + \mu P(w|C))/(|d_i| + \mu)$ is weighted by IDF and the top *fb_terms* are mixed in as $\alpha \cdot \text{original} + (1-\alpha) \cdot \text{expansion}$. The feedback prior is $P(d|R) = \lambda\,\text{uniform} + (1-\lambda)\,\text{log1p(score)}$, with λ=1 on newswire followed by a full second BM25 pass, and λ=0.25 on abstracts, where the first-pass pool of at most 250 is reranked instead. The distinction matters: a newswire document that never entered the top 250 can still be pulled in by expansion, whereas on abstracts the first-pass ordering is informative enough to keep. Two guards suppress useless expansion, one skipping it when the variance of the feedback scores falls below 10^{-6}, the other requiring a candidate term to appear in at least two feedback documents once *fb_docs* ≥ 12. The forward index supplying those counts is a per-document CSR of $tf \cdot \text{idf}$, pruned by a df band and capped per document.

2.4 Collection-adaptive parameters

`adapt_params`(N, avgdl) chooses the knobs, and it never sees query text. It interpolates between a short-document and a newswire endpoint using t derived from analysed avgdl $\in [120, 155]$, then applies discrete regimes for near-duplicate titles, short answers, forums, large abstracts, small full-text collections and newswire. The C++ `retrieve` then overlays query length, switching RM3, δ and the title bonus off for document-as-query inputs and tilting α for very short or very long queries; `retrieve_bm25` passes `apply_qlen_heuristics=false` so ablations stay pure BM25+.

The obvious worry is that this is a fingerprint table in disguise, so I tested it. The cuts are on shape rather than on a named BEIR id and sit on top of a continuous interpolation, so an unseen (N, avgdl) matching no branch keeps the blend with RM3 off. On the CORD-19 index I re-scored the 50 topics while lying to `adapt_params`: ±15% around the true pair (171,332, 109.7) stays inside the large-abstract regime and holds nDCG@10 at 0.7458, or 0.7317 when a +15% avgdl starts the b-blend. Crossing a cut on purpose, at avgdl=136 or N=99,999, drops it to the blend with RM3 off and nDCG@10 of 0.69, still comfortably above textbook BM25 at 0.636. Table 3 is the same check against real collections rather than falsified statistics: a knob shipped only if it did not collapse a differently shaped set, which is why RM3 stays off on Touché-length forums.

3 Evaluation

Every CORD-19 number below was re-run for this report on 4 September 2026 with `harness/metrics.py` (graded nDCG with $2^{\text{rel}} - 1$ gain; MAP@10, MRR and P@10 binarise rel > 0) over the 50 released TREC-COVID topics, N=171,332 and analysed avgdl=109.71. Held-out topics were never used for tuning. C++ rows share the Porter and stopword chain; Boolean, VSM and Python-BM25 rows use the S-stem indexer.

Table 1: Ranked retrievers on CORD-19 dev, k=10. Boolean AND/OR is unranked; its set sizes and recall are in §2.1, and its correctness in `tests/test_retrievers.py`.

Run	nDCG@10	MAP@10	MRR	P@10
TF–IDF cosine VSM (`lnc.ltc`, Python)	0.2815	0.0067	0.5178	0.3160
Python BM25, k_1=1.2, b=0.75, δ=0	0.5714	0.0139	0.8393	0.6360
Python BM25⊕VSM fusion (0.85/0.15)	0.5514	0.0129	0.7992	0.6300
C++ BM25, k_1=1.2, b=0.75, δ=0	0.6356	0.0155	0.8875	0.6920
C++ BM25+ (k_1=2.15, b=0.60, δ=0.80), no title	0.6323	0.0151	0.8990	0.6900
+ title β=0.60	0.6815	0.0171	0.9283	0.7380
+ RM3 (α=0.62, μ=9000, 30/40)	0.7458	0.0198	0.9589	0.8260

Two rows there deserve attention. The gap between Python BM25 at 0.5714 and C++ BM25 at 0.6356 under identical k_1, b and δ is purely the analysis chain, since the formula is the same: Porter plus a stop list is worth about 0.064 nDCG@10 here. And BM25+ without a title is very slightly *below* plain BM25, the first sign that the (k_1, b) move only pays once the title indicator is present.

Table 2: CORD-19 through the two-process harness. MAP@10 is small because these topics have hundreds of relevant documents each. Row 1 is an early snapshot from the opening leaderboard week, before the current codecs.

Phase	nDCG@10	MAP@10	MRR	Build	Latency	Index
Initial / early CORD run	∼0.735	∼0.019	∼0.93	∼3 s	∼2.9 ms	∼41 MB
This entry (re-measured)	0.7458	0.0198	0.9589	2.20 s	2.28 ms	16.6 MB

The efficiency gain is almost entirely codec work: the index shrank by roughly 60% and latency fell about 20% while quality edged up. That is the trade Simple-9 plus Elias–Fano was chosen for, since vbyte everywhere would have been marginally smaller on some lists and much slower on the ones that matter.

Table 3: The same C++ engine on every local collection. **BM25** is textbook Okapi (k_1=1.2, b=0.75, δ=0, no title, no RM3) through `retrieve_bm25_only`. **Ours** is the competition `retrieve()` with `adapt_params`. BEIR test splits; Antique test; Vaswani.

Collection	N	avgdl	#q	nDCG@10		MAP@10	P@10	RM3
				BM25	Ours			
Toy	20	7.8	6	0.8793	0.8793	0.8250	0.317	off
Cranfield	1 400	100.8	225	0.2922	0.3308	0.2891	0.271	on
CACM	3 204	33.7	52	0.5020	0.5256	0.2762	0.369	on
Vaswani	11 429	24.8	93	0.4356	0.4571	0.1637	0.381	on
SciFact	5 183	145.5	300	0.6800	0.6832	0.6353	0.090	off
NFCorpus	3 633	156.7	323	0.3240	0.3307	0.1243	0.239	on
FiQA	57 638	71.6	648	0.2476	0.2624	0.1971	0.073	off
SciDocs	25 657	115.2	1 000	0.1568	0.1654	0.0966	0.087	off
ArguAna	8 674	96.5	1 406	0.3690	0.3538	0.2313	0.074	on
Touché-2020	382 545	155.3	49	0.4014	0.4899	0.1543	0.802	off
Quora	522 931	5.8	10 000	0.7858	0.7853	0.7376	0.119	off
Antique	403 666	20.4	200	0.3486	0.4033	0.2279	0.746	off
CORD-19 / TREC-COVID (dev)	171 332	109.7	50	0.6356	0.7458	0.0198	0.826	on

Across the twelve non-CORD collections the adapted engine beats textbook BM25 on nine, ties on Toy and effectively on Quora, and loses on one. That loss is ArguAna (0.3690 → 0.3538), and the cause is legible: its queries are whole arguments, so RM3 expands toward documents restating the same stance rather than the counterargument the qrels reward, and the document-as-query overlay catches the longest of these but not all. The largest gains, Touché (+0.089) and Antique (+0.055), both come from length-regime BM25 overrides with RM3 left off rather than from expansion.

3.1 Parameter search

I swept k_1 and b with `retrieve_bm25_only`, which disables RM3, the query-length overlay and the title bonus and holds δ=0.80, so that each curve isolates a single knob (Figure 1). On the 0.3-step grid nDCG rises monotonically with k_1 through 2.8 (0.6548), while b is essentially flat across $[0.5, 0.8]$ with a nominal peak of 0.6364 at b=0.70. A denser probe on the same run ($k_1 \in [2.0, 2.4]$, everything else identical) wobbles by at most 0.006: 2.10 → 0.6337, 2.15 → 0.6323, 2.18 → 0.6370. That is 50-topic noise rather than a second protocol, so the figure plots only the grid.

I treat ±0.01 nDCG@10 as the resolution of this dev set and do not ship the BM25-only argmax, for two reasons. With RM3 enabled k_1=2.4 over-weights term frequency on abstracts, so the sweep optimum is optimal for a configuration I am not submitting; and the shipped pair has to survive Table 3, where a value fitted to 50 topics has no particular claim. The shipped $(k_1, b) = (2.15, 0.60)$ sits inside the flat b-band, and it is what the

δ=0.80 and β=0.60 additions (+0.049) and then RM3 (+0.064) stack on top of. Moving k_1 from 1.2 to 2.15 *without* the title indicator is slightly negative, and that difference is itself inside the same band.

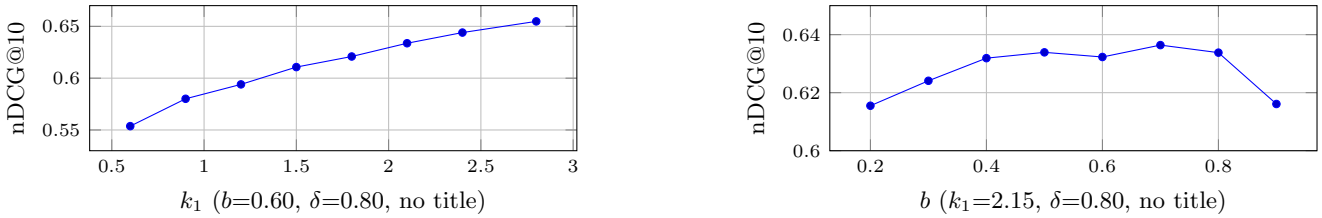


Figure 1: nDCG@10 on the 50 CORD-19 dev topics, BM25+ only, single process. The k_1 panel shows the 0.3-step grid. Off-grid, k_1=2.15 gives 0.6323, which is 0.001 below k_1=2.1 and well inside the $\pm$0.01 band implied by the dense-probe wobble of $\leq$ 0.006. The shipped pair (2.15, 0.60) is not the BM25-only peak, which is at k_1=2.8.

4 Error analysis

Five TREC-COVID topics where `retrieve()` does poorly, with re-measured nDCG@10.

1. **Query 4** (0.066), *"what causes death from Covid-19?"* The question words are in `INDEX_STOPWORDS`, so what survives is roughly {`caus`, `death`, `covid`}. Mortality language is ubiquitous in this collection and the actual mechanism the qrels reward, ARDS and cytokine response, shares no stem with the query. RM3 then makes it worse by expanding "death" into yet more mortality reviews. This is the clearest case where bag-of-words matching has no route to the answer.
2. **Query 12** (0.191), quarantine best practices. The title window rewards short overviews that happen to open with "hospital" or "quarantine", and the variance guard never fires because the first-pass scores are not near-constant, merely uniformly mediocre.
3. **Query 31** (0.266), coronavirus versus seasonal influenza. The topic is contrastive, and a bag of words cannot express a requirement of difference. Since no positions are indexed, there is no phrase or proximity evidence to fall back on either.
4. **Query 15** (0.366), how long the virus survives outside the body. It only partly overlaps the surface-stability vocabulary that the neighbouring query 16 (0.430) hits, and the expansion inherits that partial overlap.
5. **Query 49** (0.481), a long multi-facet natural-language topic. It is long but not dump-length, so the query-length overlay leaves RM3 enabled, and the expansion locks onto whichever facet dominates the feedback set.

The pattern across these is that RM3 amplifies whatever the first pass already believed, which helps when the first pass is roughly right and hurts when it is confidently wrong. Two topics that look like they should fail do not: query 42 (Vitamin D) reaches 0.948 because `vitamin` is highly selective once the bare '`d`' is dropped, and query 41 ('U.S.') reaches 0.922 because of the dotted-initial rule.

5 Final competition entry

The submitted `retrieve()` is the C++ BM25+ scorer with optional RM3, its knobs supplied by `adapt_params` from (N, avgdl) and adjusted by the query-length overlay in `engine.h`. It is not the Python fusion, which measured 0.020 nDCG@10 below plain Python BM25 and was dropped for that reason; Boolean and VSM remain in the submission as tested components rather than as part of the ranking path. The bet is that a parameterisation derived from collection shape, validated across the twelve collections in Table 3, transfers to held-out topics better than a single (k_1, b) pair fitted to 50 dev queries.

Reproducibility. `README.md` gives the full sequence. The extension is compiled during `docker build`, never inside `build_index`, so nothing is charged against build time that should not be. Worker count does not affect the output bytes, gzip timestamps are forced to 0, and ties break by score descending then doc id ascending. (Sorted JSON keys apply only to the Python `indexer.py` snapshot, not to the C++ index files.)

AI-use disclosure

LLM assistance (Cursor and Claude) was used for design discussion, code review, code generation and for drafting this report. The indexing, adaptation logic and all reported numbers were implemented and re-measured by me on the course harness.

Code provenance

No search library is used in any submitted component. Literature consulted, as papers rather than source trees: Okapi BM25 (Robertson & Zaragoza, 2009); BM25+ (Lv & Zhai, 2011); RM3 (Lavrenko & Croft, 2001) with Dirichlet smoothing (Zhai & Lafferty, 2001), where the newswire setting μ=1000, α=0.5, 10 feedback documents and 10 terms started from Anserini's published defaults and was re-tuned; Simple-9 (Anh & Moffat, 2005); Elias–Fano; front coding and vbyte (Manning et al.). Third-party code in the tree is `pybind11` (build glue) and the starter harness. `ir_datasets` is used only to fetch local corpora, not inside `retrieve()`.